

Introducción

En el desarrollo de la academia de Cidium Security, se incorporaron servicios externos con el objetivo de simular funcionalidades reales utilizadas en sistemas productivos. En particular, se realizó un primer acercamiento a la integración de pagos en línea mediante Stripe.

Estas integraciones permiten mejorar la experiencia del usuario y automatizar procesos clave dentro del sistema.

Integración con el modelo de datos

En el sistema, la gestión de pagos se modela mediante las entidades:

- **PAYMENTS**
- **PAYMENT_STUDENTS**

Estas permiten separar:

- El concepto de pago (inscripción, mensualidad)
- De la asignación individual a cada alumno

La razón principal es separar la información del pago general de la relación con los alumnos.

Es un diseño basado en normalización de bases de datos (evitar duplicidad y hacer el sistema más flexible).

Explicación de tablas de pago

1. Tabla **PAYMENTS**

Esta tabla guarda el concepto del pago, es decir:

- Qué se paga

- Cuánto cuesta
- Cuándo vence

Ejemplo:

- concept → "Inscripción"
 - amount → 500
 - currency → "MXN"
-

2. Tabla PAYMENT_STUDENTS (a quién se le asigna)

Esta tabla responde a:

¿Quién debe pagar ese concepto?

Ejemplo:

- student_id → 123
 - payment_id → inscripción
 - assigned_amount → 500
 - paid_amount → 500
-

Flujo detallado del proceso

1. Creación del pago (PAYMENTS)

El sistema registra un nuevo pago:

- concept → "Inscripción"
- amount → 500.00
- currency → "MXN"
- status → "pendiente"

Esto representa una obligación de pago, aún no pagada.

Cuando se menciona que un registro en la tabla PAYMENT_STUDENTS representa una "obligación de pago, aún no pagada", se hace referencia a que el sistema está registrando

una deuda asignada a un estudiante, pero no una transacción completada. Es decir, en ese momento solo se está indicando que el alumno tiene la responsabilidad de cubrir cierto monto, sin que el dinero haya sido procesado todavía.

Este proceso comienza cuando se define un pago en la tabla PAYMENTS, por ejemplo una inscripción o una mensualidad. Posteriormente, ese pago se asigna a uno o varios estudiantes mediante la tabla PAYMENT_STUDENTS. Al crear este registro, el sistema establece que el alumno debe realizar ese pago, por lo que generalmente se guarda con un estado como “pendiente”. En este punto, no existe todavía ninguna evidencia de que el pago haya sido realizado, solo existe la obligación.

El pago real ocurre en una etapa posterior, cuando el estudiante realiza la transacción a través de un sistema externo como Stripe. En ese momento, la plataforma procesa el cobro y, si la operación es exitosa, el sistema actualiza el registro correspondiente en la base de datos, cambiando su estado de “pendiente” a “pagado”, además de registrar información adicional como el monto cubierto o un identificador de la transacción.

Por lo tanto, la diferencia clave radica en que la creación del registro en PAYMENT_STUDENTS no implica que el pago ya se haya efectuado, sino que simplemente establece una obligación que deberá cumplirse más adelante. Esta separación permite llevar un control claro entre lo que los estudiantes deben pagar y lo que ya han pagado efectivamente.

2. Asignación al alumno (PAYMENT_STUDENTS)

Se vincula el pago con un estudiante:

- student_id → referencia al alumno
- payment_id → referencia al pago
- assigned_amount → 500.00
- paid_amount → 0
- status → "pendiente"

Aquí se define quién debe pagar y cuánto.

3. Integración con Stripe

El sistema se comunica con Stripe para crear una intención de pago (PaymentIntent).

Stripe genera:

- Un identificador único (ej: pi_123456)
 - Un estado inicial (ej: requires_payment_method)
-

4. Almacenamiento del identificador externo

Ese ID se guarda en tu sistema:

- external_reference → "pi_123456"

Este campo es CLAVE porque:

- Conecta el sistema con Stripe
- Permite consultar el estado real del pago

Cuando se procesa una transacción mediante Stripe, esta plataforma genera un identificador único, conocido como PaymentIntent ID (por ejemplo: pi_123456). Este identificador representa la operación real que ocurre fuera de tu sistema, es decir, en el proveedor de pagos.

Una vez que se obtiene este identificador, es necesario guardarlo dentro de tu base de datos, generalmente en un campo como external_reference dentro de la tabla PAYMENT_STUDENTS o PAYMENTS. Al almacenarlo, tu sistema crea un vínculo directo entre el registro interno (la obligación o el pago del alumno) y la transacción real que está siendo gestionada por Stripe.

Este campo es clave porque actúa como un puente entre ambos sistemas. Gracias a este identificador, puedes consultar en cualquier momento el estado real del pago directamente en Stripe, por ejemplo, si fue completado, rechazado o sigue en proceso. Sin este enlace, tu sistema no tendría forma confiable de verificar lo que ocurrió fuera de tu plataforma.

Además, el uso del identificador externo permite manejar eventos importantes como confirmaciones de pago, reembolsos o errores. En lugar de depender únicamente de la información interna, el sistema puede sincronizarse con Stripe y asegurar que los datos reflejen la realidad de la transacción. En resumen, guardar este identificador no es solo una

buena práctica, sino un elemento esencial para garantizar la integridad y trazabilidad de los pagos dentro del sistema.

5. Confirmación del pago

Cuando el usuario paga:

- Stripe procesa la transacción
 - Devuelve estado: succeeded o failed
-

6. Actualización en base de datos

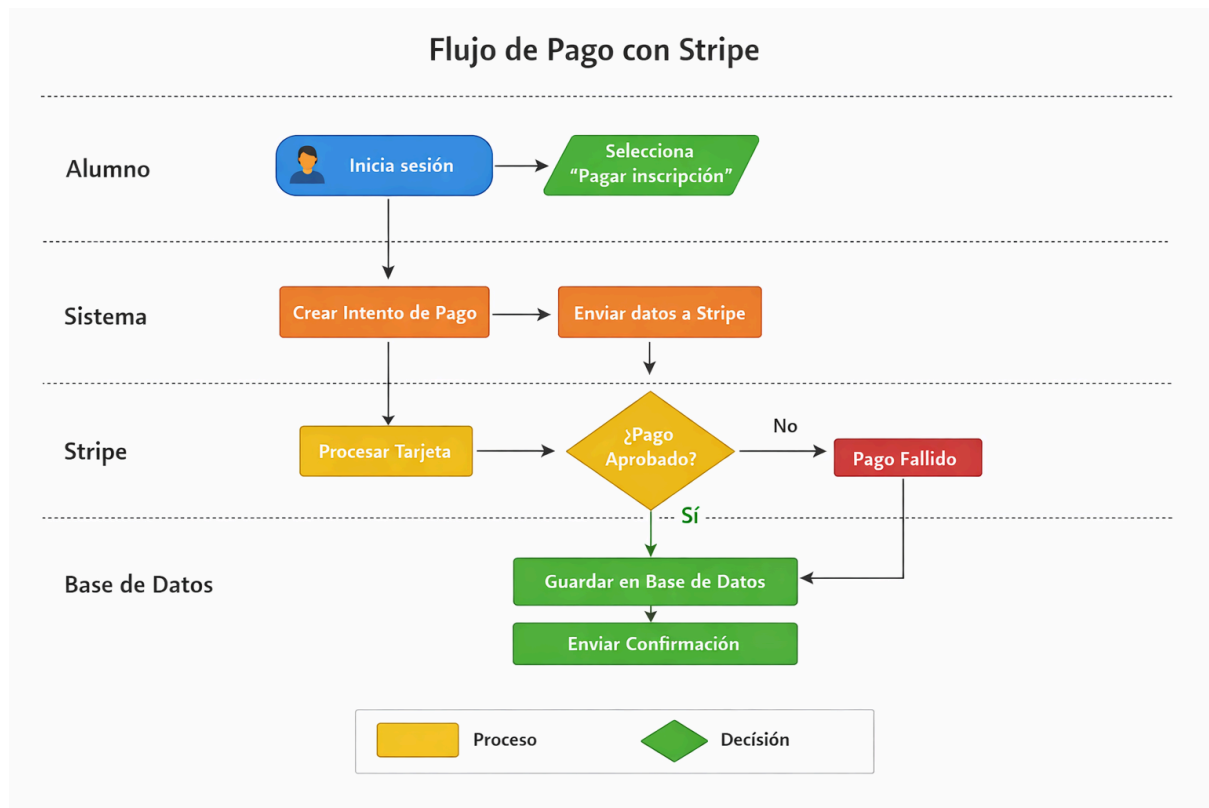
Si el pago es exitoso:

- status → "pagado"
- paid_amount → 500.00
- paid_at → fecha actual

Si falla:

- status → "fallido"
-

Diagrama de flujo de pago



El diagrama de flujo representa el proceso completo de pago dentro de la plataforma escolar, desde que el alumno inicia la acción hasta que la información queda registrada en la base de datos. Este flujo involucra la interacción entre el usuario, el sistema interno, el

Implementación endpoints

1. Configuración de Stripe

```
import stripe
```

```
stripe.api_key = "sk_test_XXXXX"
```

Explicación:

El sistema configura la conexión con Stripe utilizando una clave secreta en entorno de pruebas, lo que permite simular transacciones sin afectar datos reales.

2. Creación del PaymentIntent

```
payment_intent = stripe.PaymentIntent.create(  
    amount=50000, # en centavos  
    currency="mxn",  
    metadata={"payment_student_id": 10}  
)
```

Explicación:

El sistema crea una intención de pago (PaymentIntent) que representa una transacción en proceso. Este objeto es gestionado por Stripe y permite controlar el flujo del pago.

3. Guardar el ID en tu base de datos

```
external_reference = payment_intent.id
```

Ejemplo: "pi_123456"

Explicación:

El identificador generado por Stripe se almacena en la base de datos para vincular la transacción externa con el sistema interno.

4. Envío al frontend

```
return {  
    "client_secret": payment_intent.client_secret  
}
```

Explicación:

El sistema envía al cliente un client_secret, el cual es utilizado para completar el pago desde la interfaz de usuario.

5. Confirmación del pago (frontend)

```
stripe.confirmCardPayment(client_secret, {  
    payment_method: {
```

```
card: cardElement
}
});
```

Explicación:

El cliente utiliza la librería de Stripe para completar el pago introduciendo los datos de la tarjeta.

6. Verificación en backend

```
intent = stripe.PaymentIntent.retrieve("pi_123456")
```

```
if intent.status == "succeeded":
```

```
    # actualizar BD
```

Explicación:

El sistema consulta el estado del pago en Stripe para verificar si la transacción fue exitosa y actualizar la base de datos.

Consideraciones de implementación

Para la implementación del sistema de pagos, se tomó como referencia la documentación oficial de Stripe disponible en su sección de desarrolladores. Esta proporciona ejemplos de código y guías para integrar servicios de pago de forma segura y estructurada.

En el entorno de desarrollo, se utilizan las claves de prueba proporcionadas por Stripe, lo que permite simular transacciones sin procesar dinero real. A partir de estos recursos, se implementó la creación de intenciones de pago (PaymentIntent), que representan una transacción en proceso dentro del sistema.

Siguiendo los ejemplos proporcionados por Stripe, el backend del sistema se encarga de generar una intención de pago enviando información como el monto, la moneda y datos adicionales relacionados con el estudiante. Como resultado, Stripe devuelve un identificador único de la transacción, el cual es almacenado en la base de datos en el campo `external_reference`, permitiendo vincular el sistema interno con la plataforma externa.

Posteriormente, el cliente utiliza la información proporcionada por el servidor para completar el pago. Una vez realizada la transacción, el sistema consulta el estado del pago mediante la API de Stripe o mediante el uso de webhooks, lo que permite actualizar el estado en la base de datos.

Este enfoque basado en la documentación oficial garantiza que la integración siga buenas prácticas y estándares de seguridad, además de facilitar futuras mejoras y escalabilidad del sistema.

Mercado Pago

Mercado Pago es una plataforma de pagos muy utilizada en Latinoamérica y funciona de manera similar a Stripe, ya que ambas permiten crear transacciones externas, procesar pagos y devolver identificadores únicos que puedes almacenar en tu sistema.

Desde el punto de vista de nuestro diseño, no necesitamos cambiar nada. La forma en que está estructurado el modelo de datos, especialmente con el uso del campo `external_reference`, permite que el sistema sea independiente del proveedor de pagos. Esto significa que puedes integrar Stripe, Mercado Pago o cualquier otra API sin modificar la lógica principal ni las tablas de la base de datos.

Respecto al manejo de estados de pago, ambos sistemas funcionan de manera similar. Tanto Stripe como Mercado Pago devuelven un estado de la transacción (por ejemplo, “succeeded” o “approved”), el cual debe ser consultado por el backend para actualizar la base de datos.

La diferencia principal entre ambas plataformas radica en su enfoque. Stripe es más global y tiene una documentación muy clara, lo que facilita su uso en proyectos académicos o prototipos. Por otro lado, Mercado Pago está más adaptado al contexto latinoamericano, ofreciendo métodos de pago locales como OXXO, transferencias y pagos en efectivo, lo cual puede ser una ventaja dependiendo del tipo de usuarios del sistema.

El flujo inicia igual que con Stripe. Primero, nuestro sistema crea el registro interno en las tablas `PAYMENTS` y `PAYMENT_STUDENTS`, estableciendo la obligación de pago para el alumno con estado “pendiente”. Esto sigue siendo exactamente igual, ya que esta parte pertenece a tu lógica interna y no depende del proveedor de pagos. Una vez que tienes ese registro, es cuando entra la integración con Mercado Pago.

En lugar de crear un *PaymentIntent*, como en Stripe, en Mercado Pago se crea una **preferencia de pago**. Esta preferencia contiene la información del pago, como el monto, la descripción y los datos del usuario. Cuando haces esta solicitud a la API de Mercado Pago, la plataforma te devuelve un identificador (por ejemplo `preference_id`) y una URL de pago. Esa URL es la que se envía al frontend para que el usuario complete el pago, a diferencia de Stripe donde usas `client_secret`.

Después de recibir la respuesta de Mercado Pago, guardas el identificador en tu base de datos en el mismo campo `external_reference`. Lo que es fundamental, porque mantiene el mismo principio que es: vincular el sistema con la transacción externa.

Cuando el usuario realiza el pago, Mercado Pago procesa la transacción y luego puedes verificar el resultado. Esto se puede hacer de dos formas: consultando manualmente el estado del pago mediante la API o utilizando notificaciones automáticas (webhooks). Si el pago fue exitoso, actualizas tu tabla `PAYMENT_STUDENTS`, cambiando el estado a “pagado” y registrando el monto y la fecha. Si falla, puedes marcarlo como “fallido” o dejarlo pendiente para reintento.

Una diferencia importante en la implementación es la experiencia del usuario. Con Stripe, el pago se realiza dentro de tu aplicación usando Stripe.js. En cambio, con Mercado Pago normalmente se redirige al usuario a una página externa (checkout) donde completa el pago y luego regresa a tu sistema.

La elección entre uno u otro depende del contexto del sistema. Stripe es ideal para proyectos que requieren una integración más controlada y personalizada (porque el proceso de pago se puede manejar directamente dentro de la aplicación, dando al desarrollador control total sobre la interfaz y el flujo de la transacción.)

Ejemplo

Con Stripe podemos:

- Diseñar **nuestro propio formulario de pago**
- Mantener al usuario **dentro de tu aplicación** (no lo mandas a otra página)
- Decidir exactamente:
 - qué datos pedir
 - cómo validar
 - cuándo confirmar el pago

👉 Ejemplo: Usas `Stripe.js` y el usuario paga **sin salir de tu sistema**.

Puedes adaptar todo a tu sistema:

- UI completamente a tu estilo (colores, inputs, UX)
- Flujo de pago a tu lógica (ej: pasos, validaciones, extras)
- Integración más profunda con tu backend

No estás limitado a una pantalla externa.

Implementación con Mercado Pago

1. Configuración de la API

En el backend configuras tu token de acceso:

```
import mercadopago
```

```
sdk = mercadopago.SDK("TEST-xxxxxxxxxxxxxxxxx")
```

 Explicación: El sistema establece la conexión con la API de Mercado Pago utilizando un token de acceso en entorno de pruebas, lo que permite simular pagos sin utilizar dinero real.

2. Creación de la preferencia de pago

```
preference_data = {
```

```
  "items": [
```

```
    {
```

```
      "title": "Inscripción",
```

```
        "quantity": 1,

        "unit_price": 500.00,

        "currency_id": "MXN"

    }

],

"metadata": {

    "payment_student_id": 10

}

}
```

```
preference_response = sdk.preference().create(preference_data)

preference = preference_response["response"]
```

 Explicación: El sistema crea una preferencia de pago, que representa la transacción en Mercado Pago. Esta incluye información como el producto, el monto y datos internos del sistema.

3. Guardar el identificador en la base de datos

```
external_reference = preference["id"]
```

👉 Ejemplo: "123456789-abcde"

 Explicación: El identificador generado por Mercado Pago se almacena en la base de datos en el campo `external_reference`, permitiendo vincular el pago externo con el sistema interno.

4. Envío al frontend

```
return {  
  
    "init_point": preference["init_point"]  
  
}
```

 Explicación: El sistema envía al cliente una URL (`init_point`) donde el usuario podrá completar el pago.

5. Redirección del usuario (frontend)

```
window.location.href = init_point;
```

 Explicación: El usuario es redirigido a la página de pago de Mercado Pago, donde ingresa sus datos y completa la transacción.

6. Verificación del pago en backend

```
payment = sdk.payment().get(payment_id)
```

```
payment_info = payment["response"]
```

```
if payment_info["status"] == "approved":
```

```
    # actualizar BD
```

 Explicación: El sistema consulta el estado del pago en Mercado Pago. Si el estado es "approved", se actualiza la base de datos marcando el pago como realizado.

CONCLUSIÓN

La integración de APIs externas permite extender las capacidades del sistema, conectando el modelo de datos con servicios reales. Esto facilita la implementación de funcionalidades críticas como pagos, autenticación y comunicación, alineando la solución con estándares modernos de desarrollo de software. La integración con Mercado Pago sigue el mismo principio que Stripe, utilizando un identificador externo para vincular las transacciones con el sistema interno, lo que permite mantener una arquitectura más escalable y flexible.

Fuentes de Consulta

GitHub. Stripe samples.

(s.f.). <https://github.com/orgs/stripe-samples/repositories>

Documentación oficial de Mercado Pago:

<https://www.mercadopago.com.mx/developers>